PROYECTO UT03 - RA3: GESTIÓN DE INSTITUTOS

Acceso a Datos

22 DE FEBRERO DE 2026

IES PALOMERAS - VALLECAS
Ignacio Daniel Villacis Quishpe

Índice

1. Introducción 2
2. Herramientas y Referencias Utilizadas 2
3. Diseño de la Base de Datos 2
   3.1. Justificación del Modelo: 3
4. Desarrollo, Entidades y Persistencia 4
5. Pruebas de las Operaciones 5
   5.1. Operación 1: Listado de Asignaturas 5
   5.2. Operación 2: Listado de Centros 6
   5.3. Operación 3: Insertar Asignatura y Profesor 6
6. Parte grafica opcional 7

1. Introducción

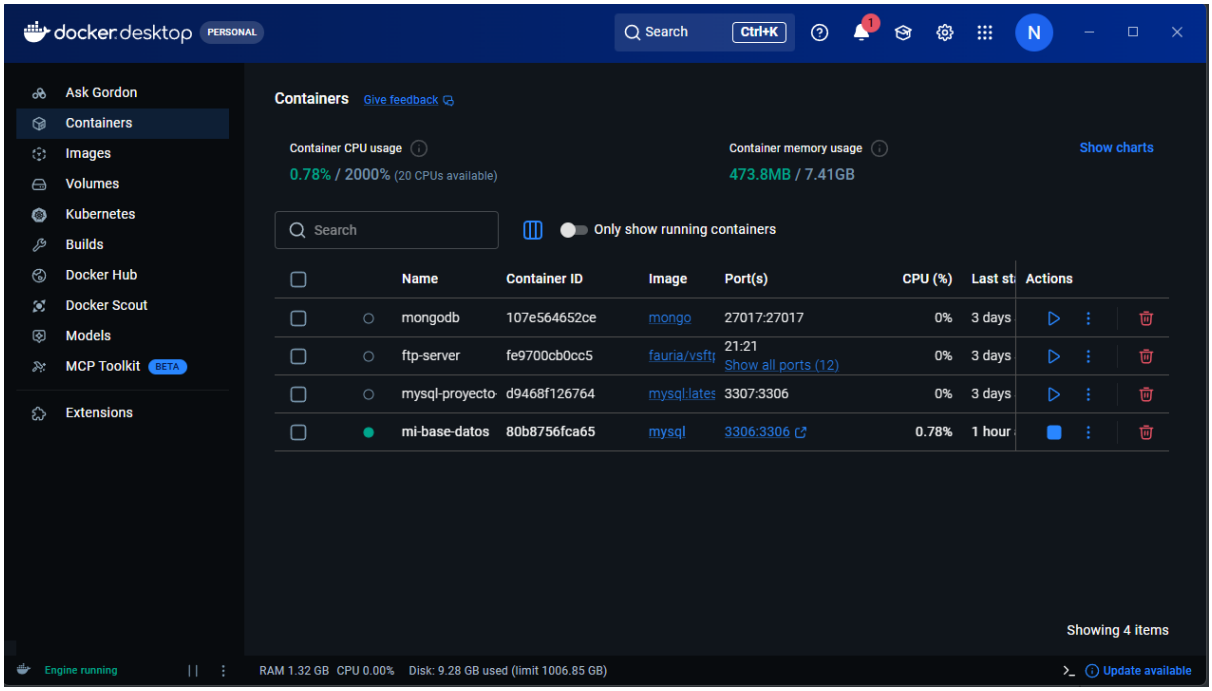
En este proyecto presento el desarrollo de la aplicación para gestionar centros educativos, tal como se pedía en los requerimientos. La idea principal era usar Java con Hibernate (JPA) para conectarme a una base de datos relacional y gestionar todo usando objetos, dejando atrás el hacer consultas SQL a mano todo el rato.

Cumplí con el requisito de tener 5 centros, 8 especialidades y 15 asignaturas, creando un script que carga todo automáticamente. Además, como punto opcional para mejorar la nota, armé una interfaz gráfica (GUI) usando JOptionPane modificado para que se vea como una ventana de escritorio de verdad y sea mucho más fácil de usar para el usuario final.

2. Herramientas y Referencias Utilizadas

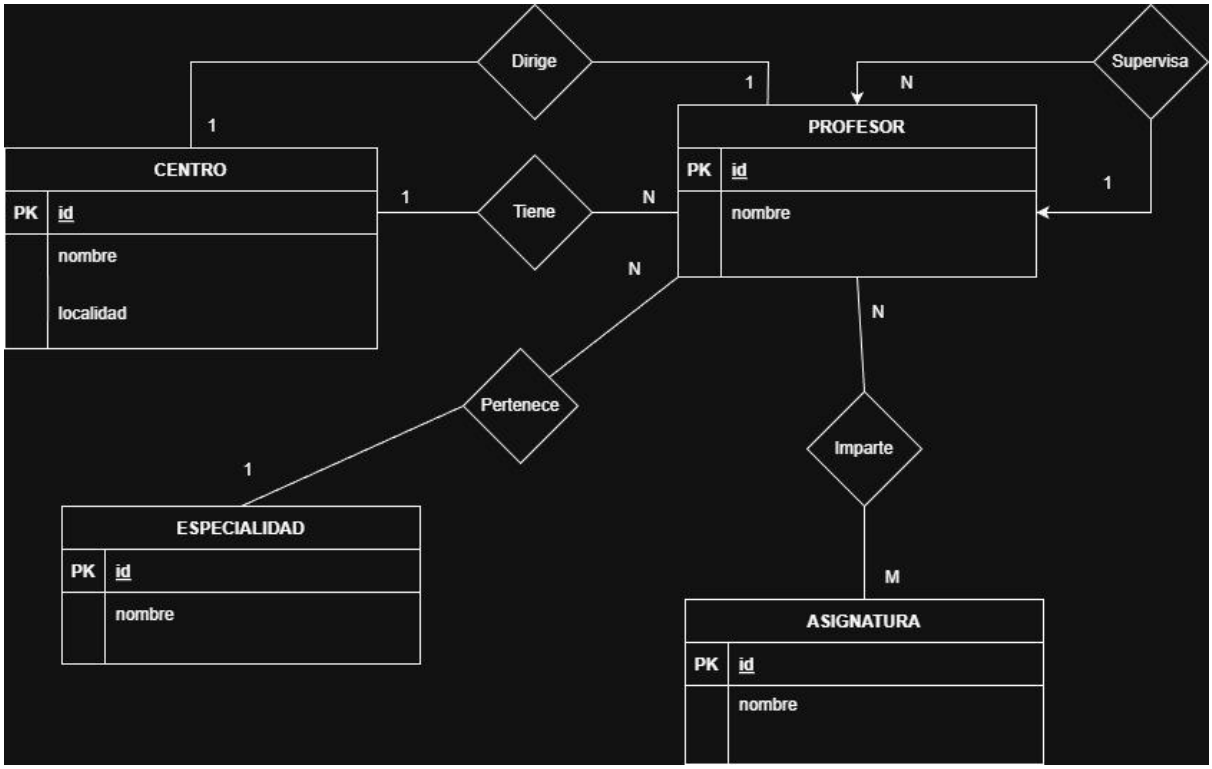
Para que esto funcione bien y a la primera, usé las siguientes herramientas basándome en los apuntes de clase y en la documentación oficial de Hibernate:

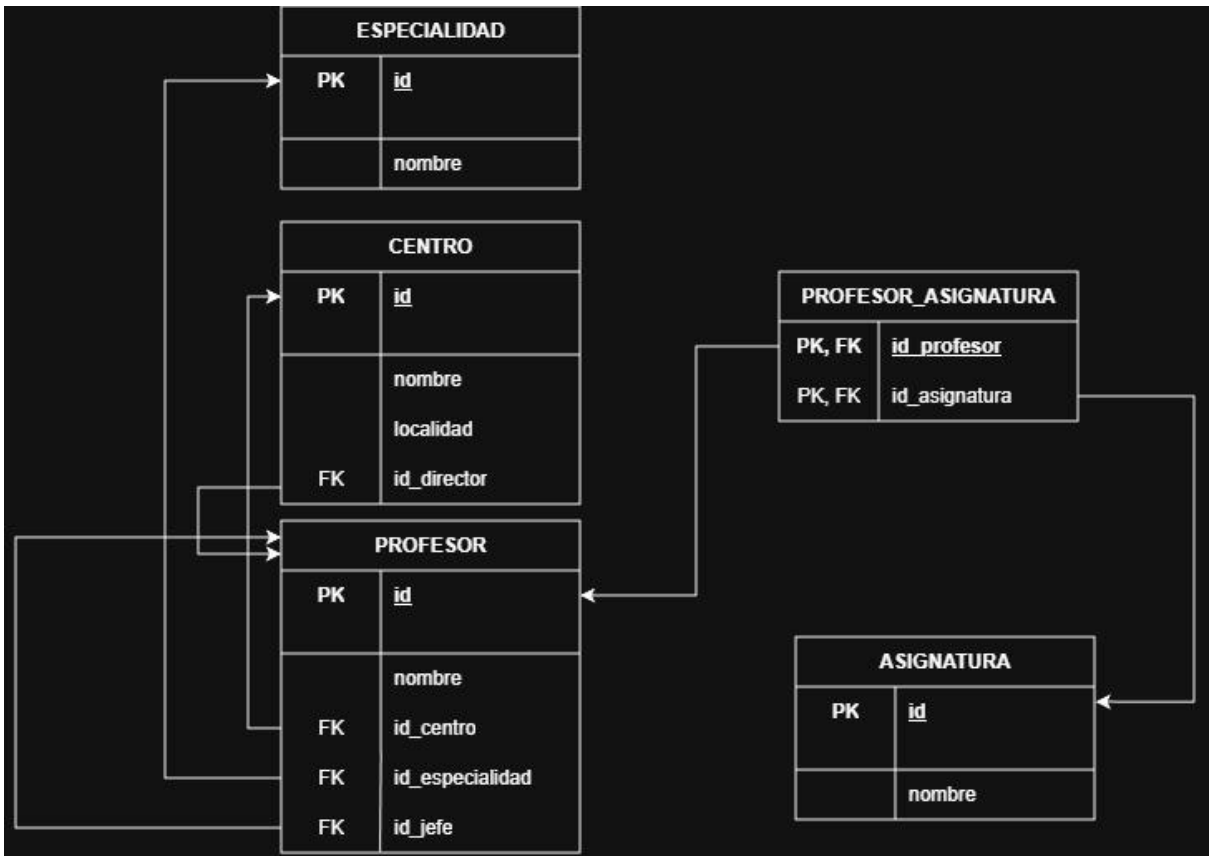
- **Lenguaje:** Java
- **IDE:** Eclipse (usando las herramientas visuales de JPA para configurar la conexión).
- **Base de Datos:** MySQL corriendo en un contenedor de Docker.
- **ORM:** Hibernate / Jakarta Persistence (JPA).



3. Diseño de la Base de Datos

Antes de meter código a lo loco, diseñé la base de datos para no perderme con las claves foráneas.





3.1. Justificación del Modelo:

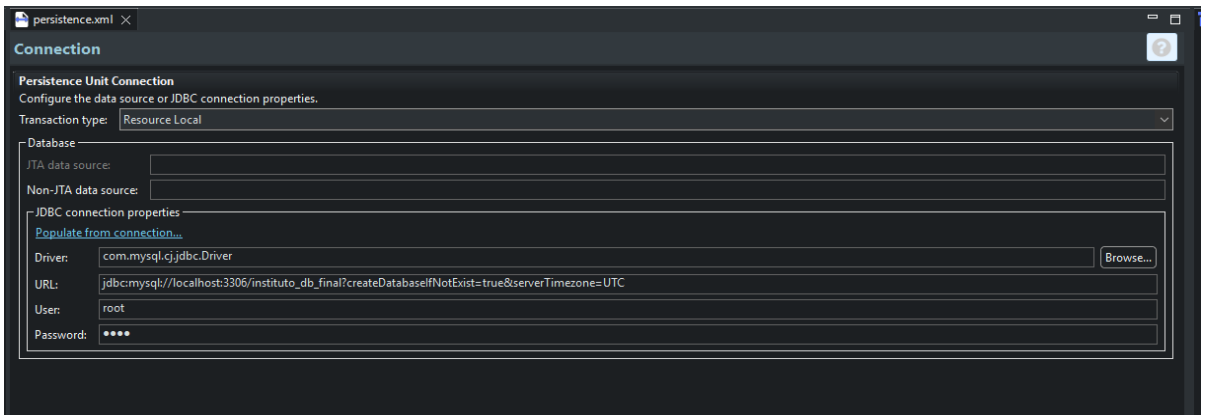
Lo armé así porque es lo que tiene más sentido lógico. Tenemos 4 entidades fuertes (Centro, Profesor, Especialidad y Asignatura). La tabla Profesor es la que más relaciones tiene: apunta a su Centro, apunta a su Especialidad y, para resolver lo del jefe de

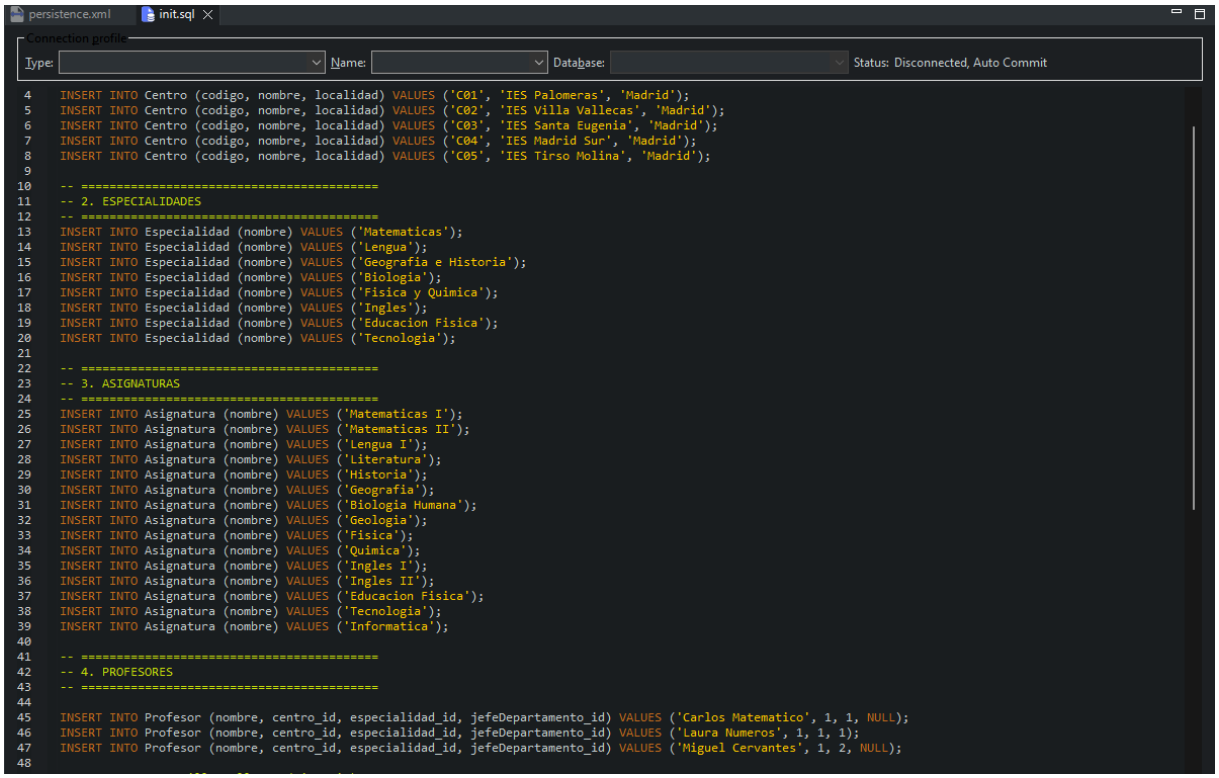
departamento, le puse una relación reflexiva hacia sí misma (id_jefe apunta al mismo Profesor). Por otro lado, como un profe da varias materias y una materia la dan varios profes, era obligatorio crear la tabla intermedia profesor_asignatura para romper la relación N:M.

4. Desarrollo, Entidades y Persistencia

Aquí está el corazón del proyecto. Usé las anotaciones de Jakarta (JPA) para mapear todas las clases. Por ejemplo, en el Profesor usé @ManyToOne para su centro y especialidad, y @ManyToMany para la lista de asignaturas.

Para la conexión a la base de datos, utilicé la pestaña gráfica "Connection" de Eclipse tal y como lo vimos en clase, configurando todo en Resource Local y conectándolo a mi MySQL. También configuré las propiedades para que las tablas se creen solas y se ejecute mi script init.sql al darle play.



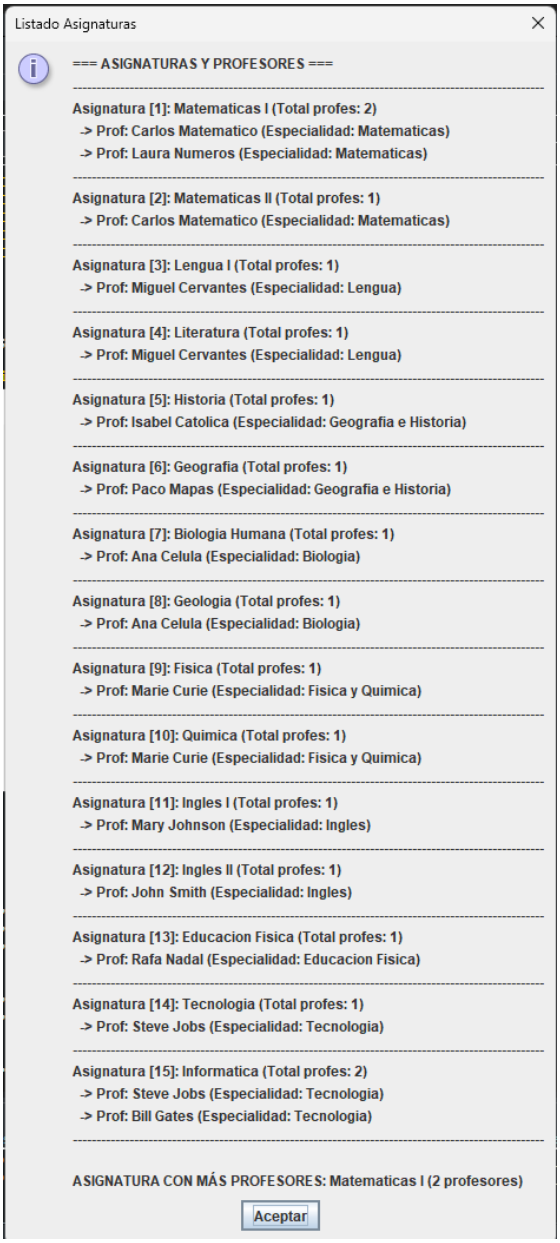


5. **Pruebas de las Operaciones**

Para demostrar que todo funciona de 10, aquí dejo las pruebas de los botones de la interfaz gráfica.

5.1. Operación 1: Listado de Asignaturas

Al darle al botón 1, el programa hace una consulta JPQL, trae las asignaturas y me muestra los profes con su especialidad. Usé un truco en la interfaz para que la ventana se adapte y se vea súper ancha y ordenada. Al final del todo, como pedía el proyecto, calcula e imprime cuál es la asignatura que más profes tiene.

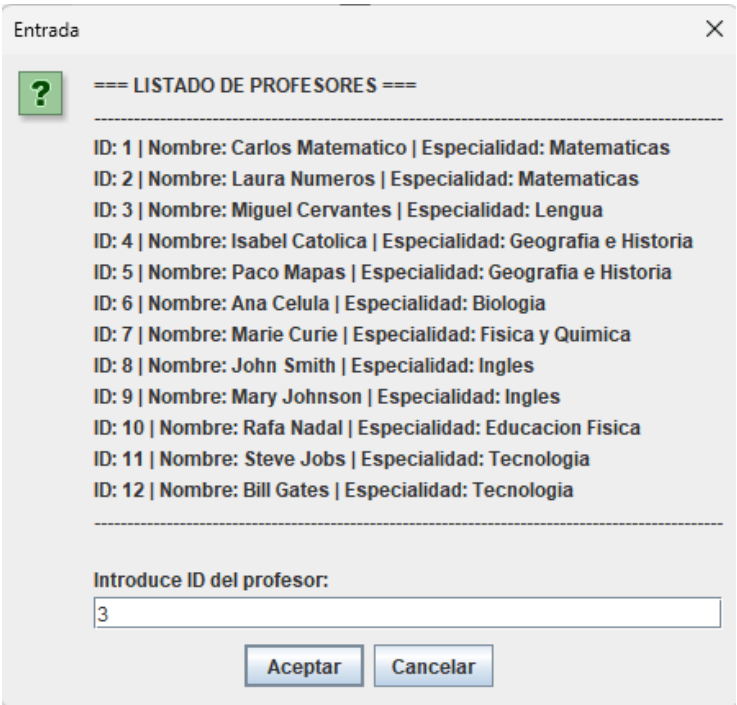


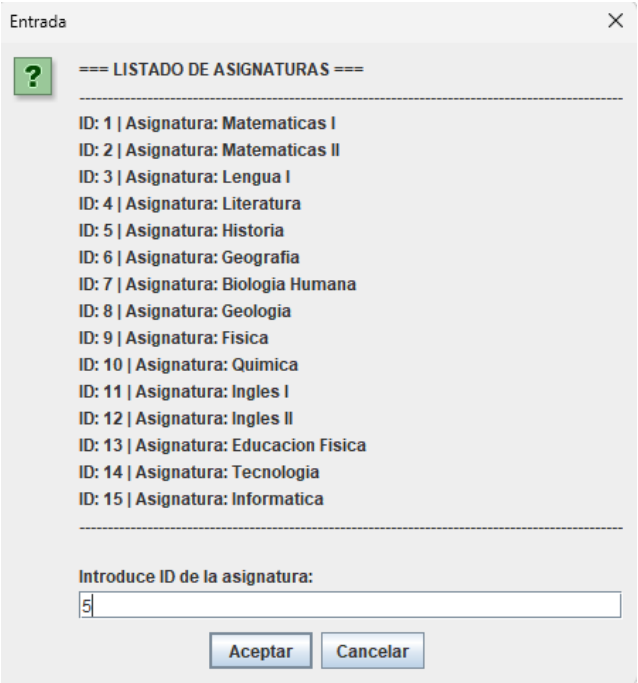
5.2. Operación 2: Listado de Centros

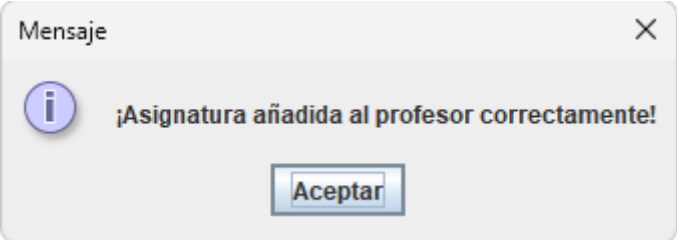
En este botón hacemos otra consulta para traer todos los centros y, recorriendo las listas, el programa saca el nombre del centro y cuántos profes en total trabajan ahí.

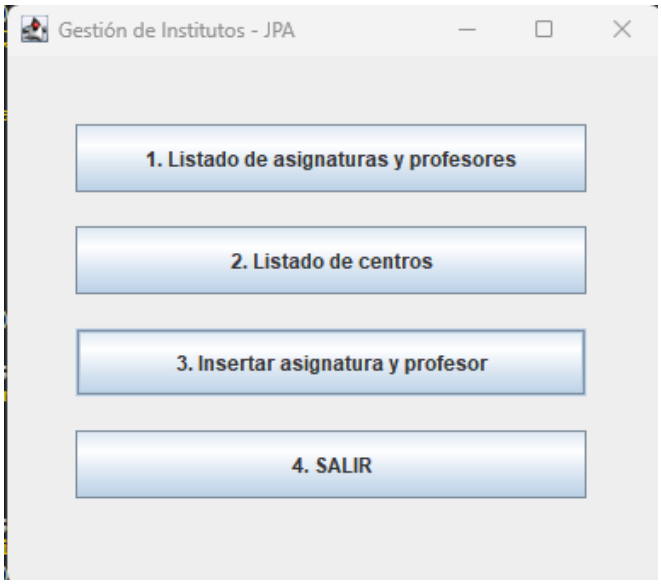
5.3. Operación 3: Insertar Asignatura y Profesor

Para hacer esto súper a prueba de tontos, primero muestro el listado de Profesores con sus IDs y especialidades para que el usuario sepa qué número poner. Hago lo mismo con las asignaturas. El programa hace los find () para verificar que existan, comprueba que el profe no dé ya esa materia para no duplicar datos, y si todo está correcto, hace el em.getTransaction().commit() para guardar en la base de datos real. Si el usuario mete una letra en vez de un número, salta una ventana de error capturada por un try/catch.







6. Parte grafica opcional

Como mejora opcional de la interfaz gráfica, decidí no hacer una simple impresión por consola negra, sino integrar todo en un menú hecho con JFrame y botones. Para los cuadros de diálogo, modifiqué los mensajes del JOptionPane usando StringBuilder y separadores para que la experiencia del usuario sea mucho más limpia y profesional.